Certified RDE Certification — Hands-On Project

Smart Backlog Assistant: Problem Definition & Solution Design

Required Steps 1 & 2 Deliverable • Configurable Multi-Agent AI Engineering Solution

1. Problem Definition (Required Step 1)

The Engineering Problem

Forward Deployed Engineers (FDEs) and Agile engineering teams lose hours every sprint manually converting unstructured inputs—stakeholder meeting notes, incident postmortems, and compliance PDFs—into sprint-ready backlog tickets. Manual grooming and naive zero-shot AI prompts routinely introduce four critical failure modes:

- **Duplicate Work (DUPLICATE):** Re-creating user stories that already exist in the active backlog.
- **Missed Scope Updates (UPDATES_EXISTING):** Failing to link new operational constraints to the existing tickets they modify.
- **Undetected Architectural Conflicts (CONFLICT):** Accepting compliance mandates (e.g., encryption rules) that contradict existing implementation stories without flagging human review.
- **Requirement Hallucinations:** Guessing unspecified SLAs, retention windows, or schemas instead of surfacing ambiguities for stakeholder sign-off.

Target Use Cases (3 Realistic Scenarios)

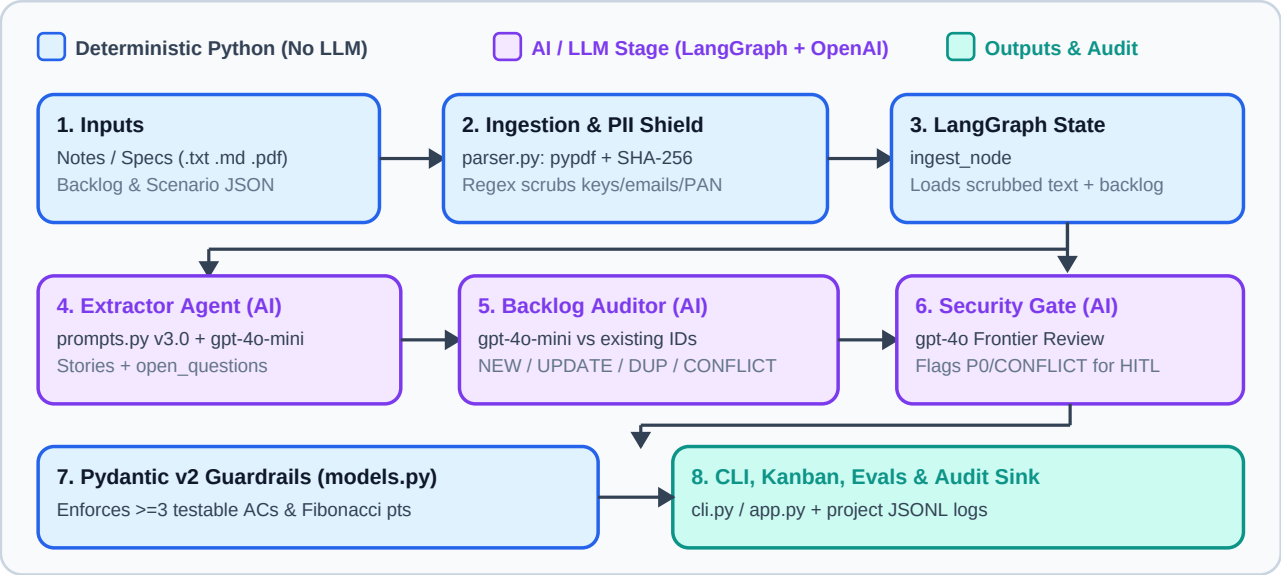
Use Case	Sample Input	What the Solution Delivers
1. Sprint Planning & Stakeholder Sync	01_sprint_planning_notes.txt	Extracts federated OIDC/SAML 2.0 SSO (NEW), catches a duplicate CSV export request (FIN-101 -> DUPLICATE), and links exponential backoff + SQS DLQ to FIN-104 (UPDATES_EXISTING).
2. SEV-1 Incident Postmortem Triage	02_incident_postmortem.md	Translates a cascading payment-gateway timeout postmortem into an adaptive circuit-breaker story (NEW) and tightens timeout/bulkhead limits on FIN-105 (UPDATES_EXISTING).
3. Security & Compliance PDF Audit	03_pci_compliance_spec.pdf	Parses a PCI-DSS v4.0 PDF specification, flags a direct encryption conflict against plaintext Redis caching in FIN-106 (CONFLICT + HITL Gate), and extracts an S3 Object Lock audit story (NEW).

AI Tools Used to Refine the Problem Statement

Used AI pair-programming to stress-test the problem statement against real FDE workflows—ensuring the scenarios exercised all four backlog alignment states (NEW, UPDATES_EXISTING, DUPLICATE, CONFLICT), local PII/secret redaction, and an anti-hallucination sink (open_questions) rather than simple text summarization.

2. Solution Design (Required Step 2)

Architecture Diagram: Components, Data Flow & Where AI Is Used



Data Flow & Separation of Responsibilities

- **Deterministic Pre-Processing (parser.py):** Reads .txt, .md, and .pdf (via pypdf), computes a SHA-256 fingerprint, and scrubs emails, API keys, PANs, and SSNs locally with regex before any external API call.
- **Multi-Stage AI Pipeline (engine.py — LangGraph + ChatOpenAI):** *extractor_agent* (gpt-4o-mini) generates structured stories and routes missing specs to open_questions; *backlog_auditor* (gpt-4o-mini) classifies NEW / UPDATES_EXISTING / DUPLICATE / CONFLICT against existing JSON IDs; *security_reviewer* (gpt-4o) applies Human-in-the-Loop (HITL) gating on P0/CONFLICT items. Includes a deterministic offline replay fallback when no API key is present.
- **Deterministic Guardrails & Delivery (models.py, cli.py, app.py):** Pydantic v2 enforces >=3 testable acceptance criteria and Fibonacci sizing before exporting Markdown/JSON reports, syncing the Jira-style Kanban board, and logging FinOps/audit telemetry.

Approach to Prompt Design (v1 -> v3.0-fde-configurable)

- **Iteration 1 (v1 — Zero-Shot):** Asked for JSON stories directly. Failed due to priority inflation, vague acceptance criteria, duplicate tickets, and invented SLAs.
- **Iteration 2 (v2 — Role + Rubric):** Added a Staff FDE persona, P0–P3 priority rubric with mandatory priority_rationale, and Fibonacci points [1, 2, 3, 5, 8, 13].
- **Iteration 3 (v3.0-fde-configurable in prompts.py):** Parameterized domain variables via ScenarioConfig (scenario_config.json / presets), injected existing_backlog.json for 4-state alignment, and added the open_questions anti-hallucination sink paired with Pydantic-enforced Structured Outputs.

AI Assistance Used in the Design Phase

Used AI pair-programming to co-design the Pydantic v2 Structured Output schema (LLMExtractionPayload / BacklogAnalysisResult), calibrate the 12-case golden evaluation dataset (evals/golden_dataset.json), and design cost-optimized multi-tier model routing (gpt-4o-mini for extraction/auditing; gpt-4o reserved for security/judge gates).